Contents

Writable storage beside an Alpine live environment **1**
What the existing dd command does 1
The desired layout 1
Existing raw-written USB: inspect its layout first 2
A planned single-drive alternative 2
Saving Alpine state is a separate choice 3

Writable storage beside an Alpine live environment

This accompanies section 5 of `/home/heini/Downloads/arch-root-on-zfs-luks2.md`. It does not change that runbook's disk-write commands or require ZFS.

What the existing dd command does

`dd if=alpine-extended-...iso of=/dev/VERIFIED_USB` copies the image bytes, including its hybrid boot and partition structures, to the whole device. It writes the image length; it neither expands the image filesystem nor turns the remaining capacity into a filesystem. The ISO9660 filesystem remains read-only.

For example, a small ISO on a 64 GB USB can leave most of the physical capacity beyond the image. Those bytes may still contain old data; they are not necessarily zeroed or securely erased. A partition/filesystem is needed before that space is ordinary writable storage.

Neither `dd bs=` nor `count=` adds usable storage to the live system. Appending zeros or changing mount options does not resize ISO9660.

The desired layout

Conceptually:

```
USB drive
  Alpine boot files / live image
  Writable Linux data filesystem
    arch-bootstrap/             retained Arch userspace
      var/cache/pacman/pkg/     retained Arch package downloads
    notes/
    apk-cache/                  optional Alpine package cache
```

Use a Linux filesystem such as ext4 for the **extracted** Arch bootstrap. FAT/exFAT can hold the compressed archive and ordinary notes, but cannot represent a normal Linux root's permissions, symlinks, and special files.

Existing raw-written USB: inspect its layout first

It may be possible to add a data partition entirely after the image, but the exact command depends on the embedded table. Alpine does not document a universal append-partition procedure for all its hybrid ISO versions. Do not assume the free space is partition 2 or 3, or accept a partition tool's offer to repair/convert GPT without understanding that image.

From another booted system, identify the actual USB and inspect it:

```
lsblk -o NAME,PATH,SIZE,MODEL,TRAN,TYPE,FSTYPE,PTTYPE,MOUNTPOINTS
sudo fdisk -l /dev/VERIFIED_USB
sudo wipefs --no-act /dev/VERIFIED_USB
```

`wipefs --no-act` only reports signatures here; do not omit `--no-act` in any write-mode adaptation. These commands do not partition or format.

The evidence must establish the device identity, image partition ranges, table type(s), and unused tail. Any later data partition must preserve the image's boot structures and every occupied sector. Generic `mklabel`, `sgdisk -e`, `wipefs -a`, and guessed `mkfs /dev/sdXn` commands are not appropriate here. Do not modify the partition table of the medium currently providing the live environment while it is in use.

Until that layout is known, keep the direct-written USB as-is and use a second writable device. There is no need to alter its boot image merely to bootstrap Arch.

A planned single-drive alternative

Alpine documents a **customizable boot device**, prepared with ordinary partitions/filesystems and `setup-bootable` rather than raw-writing the ISO. It provides writable boot storage and supports diskless configuration backups and APK caching.

For a reusable installation USB, a practical design is a FAT32 boot partition containing Alpine's boot files plus an ext4 work partition occupying the remaining space. The FAT32 partition suits the documented boot workflow; the ext4 partition holds the extracted Arch userspace. Actual partition/bootloader commands must match the machine's firmware and the selected target. This is a rebuild, not an in-place conversion of ISO9660; make it from another working boot medium with copies of the required files and data elsewhere.

`setup-bootable` accepts an ISO file or mounted source directory and copies its contents to a prepared writable destination. Follow Alpine's Create a Bootable Device procedure for the complete boot setup.

After the work partition exists and is mounted at root-owned `/mnt/work`:

```
findmnt --mountpoint /mnt/work -o TARGET,SOURCE,FSTYPE,OPTIONS
df -h /mnt/work
```

```
bash ./arch-bootstrap.sh /mnt/work/arch-bootstrap
```

After reboot, mount the same data filesystem and use:

```
bash ./arch-bootstrap.sh --enter /mnt/work/arch-bootstrap
```

When finished, exit the Arch shell, move any host shells out of `/mnt/work`, and unmount the work filesystem normally before removal. If it is busy, inspect holders instead of forcing removal.

Saving Alpine state is a separate choice

The bootstrap directory already persists as ordinary files on the data filesystem. It does not need Alpine's own persistence mechanism.

Alpine diskless configuration can be saved as an `.apkovl` archive using `lbu`, and APK files can be cached on writable storage. `setup-lbu` and `setup-apkcache` configure those choices. An `lbu commit` saves the selected configuration; it does not automatically preserve every change under `/`. Keep large bootstrap roots and package caches outside the backup archive.

References:

- Alpine: Create a Bootable Device
- Alpine: Diskless Mode
- Alpine: Local APK cache
- Alpine: local backup
- Alpine: configuration scripts